

PLAN ASSURANCE QUALITÉ

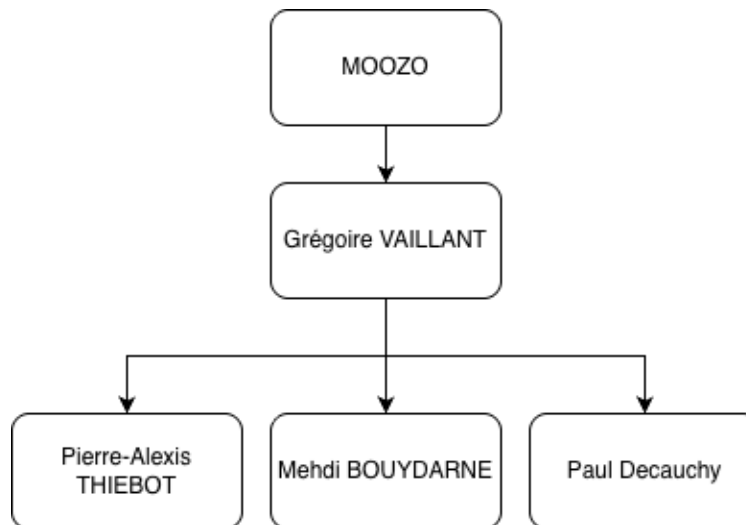
Table of Contents

Structure organisationnelle du projet (OBS)	4
Clarté hiérarchique	4
Cartographie des compétences	4
Définition des rôles	5
Mehdi BOUYDARNE : Développeur opérationnel	5
Paul DECAUCHY : Développeur backend	5
Pierre-Alexis THIEBOT : Développeur IA / ALGO	5
Grégoire VAILLANT : PO / Développeur full stack	6
NORMES	7
Git	7
Branch	7
Commit	7
Code	8
GO	8
Python	9
React	9
MÉTHODOLOGIE	10
OUTILS	11
Git / Github	11
Jira	11
OneDrive	11
Google drive	11
WhatsApp	11
Slack	11
Figma	11
TESTS	13
Tests Unitaires (TU)	13
Tests d'Intégration	13
Tests End-to-End (E2E)	13
Critères de Qualité et Couverture	13
Processus d'Exécution (Workflow)	13
DOCUMENTATION ONBOARDING	15
Readme	15

Contributing	15
CHOIX TECHNOLOGIQUES	16
Frontend – web	16
React TS	16
Frontend - mobile	17
React Native	17
Backend (API)	17
Golang	17
Ogen	18
Backend (IA)	18
Python	19
PyTorch	19
Base de données.....	19
MongoDB	19
Déploiement.....	19
Docker	19

Structure organisationnelle du projet (OBS)

Clarté hiérarchique



L'équipe adopte une structure volontairement horizontale afin de favoriser la communication, l'autonomie et la réactivité.

Chaque membre est responsable de son périmètre technique tout en contribuant aux décisions collectives.

Cependant, afin d'éviter tout blocage décisionnel :

- Grégoire VAILLANT occupe le rôle de décisionnaire final.
- En cas de désaccord ou de nécessité de trancher rapidement, la décision finale est prise par Grégoire.
- Cette organisation permet de maintenir un équilibre entre collaboration ouverte et efficacité stratégique.

Cartographie des compétences

La cartographie des compétences est maintenue dans le fichier [MOOZO.xlsx](#), dans l'onglet dédié aux compétences de l'équipe.

Ce fichier permet d'identifier rapidement :

- Les compétences principales de chaque membre

- Les domaines d'expertise
- Les domaines secondaires
- Les besoins d'accompagnement ou de montée en compétence
- Les personnes référentes par périmètre technique

Définition des rôles

Mehdi BOUYDARNE : Développeur opérationnel

Après 3 ans de parcours généraliste en informatique, je me suis spécialisé en Cybersécurité et Cloud.

En alternance au sein de la cellule cyber d'une PME, je travaille sur l'automatisation et l'industrialisation des déploiements, la gestion d'infrastructure et la surveillance (mise en place d'une infrastructure de collecte/monitoring, construction d'un SIEM). Je contribue également à la sécurisation de l'infrastructure (durcissement, contrôle des accès, bonnes pratiques d'exploitation).

Paul DECAUCHY : Développeur backend

Développeur backend spécialisé en Go, j'évolue actuellement chez Ubisoft au sein du Lab, une entité dédiée à l'exploration de nouveaux usages technologiques.

Je conçois des systèmes backend distribués en appliquant les principes du Domain-Driven Design (DDD), une approche qui consiste à structurer l'architecture autour du métier afin de garantir cohérence et évolutivité.

Je travaille notamment sur l'implémentation de workflows asynchrones via Uber Cadence, ce qui implique la gestion de processus complexes et de longue durée, avec des mécanismes de reprise automatique en cas d'erreur et de maintien de la cohérence des données.

J'interviens également sur l'intégration des données avec les serveurs de jeu, en veillant à assurer performance, fiabilité et qualité logicielle.

Pierre-Alexis THIEBOT : Développeur IA / ALGO

Mon profil plutôt technique. Après 3ans en développement de jeux-vidéo où j'ai développé de solides compétences en algorithmique et en optimisation, je suis actuellement en spé cybersécurité, ce apporte une compréhension des architectures, des performances et des enjeux liés à la sécurité des systèmes.

Je travaille aussi dans une startup développant un SaaS de simulation de campagnes de phishing à des fins de sensibilisation, où je touche à tout sauf au front.

J'y ai notamment développé un **evil proxy** (proxy intermédiaire capable d'intercepter et de relayer des flux pour analyser des mécanismes d'authentification dans un cadre de test), des systèmes d'envoi de campagnes de phishing simulées ainsi que des mécanismes de sécurisation. Cette expérience renforce mon intérêt pour les systèmes intelligents, l'automatisation et la compréhension fine des architectures techniques.

Même si je ne suis pas en spé IA, l'IA et la conception d'algorithmes sont des domaines qui me passionnent particulièrement.

Grégoire VAILLANT : PO / Développeur full stack

Mon profil est hybride, j'ai une formation initiale en école de commerce à emlyon business school, puis une spécialisation technique en développement informatique à EPITECH. Cette double formation me permet de comprendre à la fois les enjeux stratégiques d'un projet (vision produit, priorisation, cohérence globale) et ses contraintes techniques concrètes.

Au cours de mon alternance chez Quiz Room, j'ai été impliqué à la fois dans le développement full stack (MERN, API, AWS S3, gestion serveur) et dans des échanges organisationnels et fonctionnels. Cette position intermédiaire m'a naturellement amené à faire le lien entre les besoins métier et l'implémentation technique.

NORMES

Les normes de développement ont pour objectif de garantir :

- La lisibilité du code
- La maintenabilité du projet
- La cohérence entre les contributions
- La réduction des erreurs
- La simplification des revues de code
- La qualité des livrables

Ces normes portent principalement sur le contrôle de version, la structure des branches, les conventions de validation et les normes de codage pour chaque langage de programmation utilisé dans le projet.

Git

Le projet utilise Git pour la gestion de version. Le dépôt est hébergé sur GitHub afin de faciliter la collaboration, la gestion des pull requests, les revues de code et l'intégration continue.

Aucun développement ne doit être réalisé directement sur la branche principale.

Branch

Les branches utilisées sont les suivantes :

- **main** : version stable du projet
- **develop** : branche principale de développement
- **feature/nom-feature** : développement d'une nouvelle fonctionnalité
- **fix/nom-fix** : correction d'un bug
- **docs/nom-documentation** : modification documentaire
- **refactor/nom-refactor** : refactorisation sans changement fonctionnel

Toute fusion vers **develop** doit passer par une pull request validée par au moins un autre membre de l'équipe.

Commit

Les messages de commit suivent une convention inspirée de Conventional Commits :

- feat: ajout d'une fonctionnalité ;
- fix: correction d'un bug ;

- refactor: modification interne sans changement fonctionnel ;
- docs: modification de la documentation ;
- test: ajout ou modification de tests ;
- chore: tâche technique sans impact fonctionnel direct.

Exemples :

- feat: ajout de la gestion du plan de table
- fix: correction du calcul de position des invités
- docs: mise à jour du guide de contribution

Code

Les règles communes suivantes s'appliquent à l'ensemble du projet :

- Utiliser des noms explicites pour les variables, fonctions, composants et fichiers
- Limiter la longueur des fonctions
- Appliquer le principe de responsabilité unique
- Éviter la duplication de code
- Gérer explicitement les erreurs
- Écrire des commentaires uniquement lorsque le code n'est pas immédiatement compréhensible
- Privilégier la lisibilité à la complexité inutile
- Maintenir des fichiers courts et cohérents

Le formatage automatique est obligatoire lorsque l'écosystème du langage le permet.

GO

Les règles Go suivantes sont appliquées :

- Formatage obligatoire avec gofmt ;
- Noms de packages courts, en minuscules, sans underscore ;
- camelCase pour les variables, fonctions et méthodes non exportées ;
- PascalCase pour les structures, interfaces, fonctions et méthodes exportées ;
- Acronymes usuels conservés en majuscules dans les noms : ID, URL, HTTP, API, JSON, SQL ;
- Imports placés en haut du fichier (bibliothèques standard, bibliothèques externes, modules du projet)
- Séparation des imports entre bibliothèque standard, dépendances externes et modules internes ;

- Utilisation de structs et interfaces plutôt que de classes ;
- Préférence pour la composition plutôt que l'héritage ;
- Commentaires GoDoc obligatoires pour les éléments exportés ;
- Gestion systématique des erreurs avec `if err != nil` ;
- Ajout de contexte aux erreurs avec `fmt.Errorf("...: %w", err)` lorsque pertinent ;
- utilisation de `context.Context` en premier argument pour les opérations longues, réseau ou annulables ;
- un package doit avoir une responsabilité claire ;
- éviter les packages génériques de type `utils` ;
- tests placés dans des fichiers `_test.go` avec le package `testing` ;
- gestion des dépendances avec `go mod`.

Python

Les règles Python suivantes sont appliquées :

- `snake_case` pour les variables, fonctions et méthodes ;
- `PascalCase` pour les classes et exceptions ;
- `UPPER_CASE` pour les constantes ;
- `_leading_underscore` pour les éléments à usage interne ;
- méthodes spéciales Python écrites avec la convention dunder, par exemple `__init__` ou `__str__` ;
- acronymes usuels conservés en majuscules dans les noms : ID, URL, HTTP, API, SQL, DB ;
- imports placés en haut du fichier ;
- séparation des imports entre bibliothèque standard, dépendances externes et modules internes ;
- docstrings obligatoires pour les classes et fonctions publiques ;
- fonctions courtes et à responsabilité unique ;
- gestion des erreurs avec des blocs `try/except` ciblant des exceptions précises ;
- utilisation de `pytest` pour les tests.

Remarque : l'indentation Python doit être standardisée. Il est recommandé d'utiliser quatre espaces, conformément à la convention majoritaire de l'écosystème Python. Si l'équipe maintient une règle différente, elle doit être explicitement justifiée et appliquée automatiquement par l'outillage.

React

Les règles React et TypeScript suivantes sont appliquées :

- Utilisation de TypeScript pour typer les props, états, retours d'API et modèles métier
- Composants courts et réutilisables
- Séparation claire entre composants UI, logique métier, hooks, services API et types
- Organisation moléculaire des composants lorsque cela est pertinent
- Nommage explicite des composants en PascalCase
- Nommage des hooks avec le préfixe use
- Formatage automatique avec Prettier
- Tests de composants avec Jest et React Testing Library
- Éviter la logique métier complexe directement dans les composants d'interface.

MÉTHODOLOGIE

Pour le développement du projet Moozo, l'équipe a choisi d'utiliser la méthode Scrum, issue des méthodes Agile software development. Cette approche est particulièrement adaptée aux projets logiciels nécessitant des évolutions régulières et des ajustements en fonction des retours des utilisateurs ou des nouvelles contraintes techniques.

Contrairement à la méthode Waterfall model, qui repose sur une progression linéaire et séquentielle des phases (analyse, conception, développement, test), Scrum permet une organisation plus flexible basée sur des cycles courts appelés sprints. Cette approche facilite l'adaptation aux changements et permet de livrer régulièrement des fonctionnalités opérationnelles.

Dans le cadre du projet Moozo, l'utilisation de Scrum permet notamment : d'avancer progressivement sur les différentes fonctionnalités (gestion des invités, plan de table, partage de photos, etc.) de tester régulièrement les fonctionnalités développées d'identifier rapidement les problèmes techniques ou fonctionnels

L'équipe prévoit d'organiser un sprint par semaine, avec une réunion de suivi hebdomadaire permettant de faire le point sur l'avancement du projet, les tâches réalisées et les difficultés rencontrées. Cette réunion permet également de planifier les objectifs du sprint suivant.

Plusieurs livrables et outils de suivi seront produits afin de structurer l'avancement du projet :

- Compte-rendu de réunion (CR) afin de garder une trace des décisions et des points discutés
- Backlog produit listant l'ensemble des fonctionnalités à développer

- Burndown chart, utilisé pour suivre l'avancement du sprint et visualiser la quantité de travail restante
- Documentation technique, décrivant l'architecture et les choix technologiques du projet

Ces éléments permettent de suivre l'avancement du projet de manière structurée et de garantir une bonne coordination entre les membres de l'équipe.

OUTILS

Git / Github

Utilisation de Git et GitHub pour le versionnement du code et l'organisation des différents projets. Le développement se fait sur une branche créée depuis la branche develop lorsque le ticket passe en In Progress. Une pull request est ensuite ouverte avec au minimum une review externe avant validation, et les tests unitaires doivent être exécutés et validés avant le merge.

Jira

Jira (software) est utilisé pour la gestion des tickets, des user stories et du suivi de l'avancement des tâches. Le workflow classique est : To Do → In Progress → Review → Done. Le passage en Done est conditionné par la validation du code et des tests. Le merge d'une pull request entraîne la fermeture automatique du ticket lorsque l'intégration est configurée.

OneDrive

Microsoft OneDrive sert au stockage et au partage des documents administratifs, cahiers des charges, rapports et fichiers bureautiques au format Word ou similaire. L'objectif est d'avoir un référentiel documentaire unique et accessible à l'équipe.

Google drive

Microsoft OneDrive sert uniquement au stockage des graphiques fait avec draw.io

WhatsApp

WhatsApp est utilisé pour la communication rapide entre les membres de l'équipe pour les questions opérationnelles, la coordination ou les informations urgentes.

Slack

Slack sera utilisé pour intégrer un bot chargé de publier automatiquement des informations liées au dépôt de code ainsi qu'aux environnements de développement et de production (notifications de commits, déploiements, incidents, etc.).

Figma

<https://www.figma.com/design/gDuCBWyCdKWmXbWtsRiqmB/Moozo?node-id=0-1&t=9C3leBeUPA4HxRqC-1>

Figma est utilisé pour la conception des maquettes, prototypes d'interface et validation des écrans avant la phase de développement. Les designs y sont partagés pour recueillir les retours des parties prenantes.

TESTS

La stratégie de tests vise à garantir :

- La fiabilité des fonctionnalités ;
- La non-régression ;
- La cohérence entre les modules ;
- La validation des règles métier ;
- La qualité des API ;
- La robustesse des parcours utilisateur principaux.

Tests Unitaires (TU)

- Backend (Go) : Utilisation du package natif testing. Les fichiers dans repositories et dans domain doivent etres accompagnes de leurs tests.
- Backend (IA Python) : Utilisation de pytest pour valider les algorithmes de matching de reconnaissance faciale (à voir).
- Frontend (React) : Utilisation de Jest et React Testing Library pour tester la logique des composants.

Tests d'Intégration

- Framework: github.com/ovh/venom
- Objectif : Vérifier la communication entre les différents modules
- Périmètre : Validation des endpoints des APIs, vérification de la persistance des données dans MongoDB et bon fonctionnement des middlewares.

Tests End-to-End (E2E)

- Objectif : Simuler des user stories.
- Outils : Utilisation de Playwright (à voir) pour automatiser les tests sur les navigateurs web et simulateurs mobiles.

Critères de Qualité et Couverture

Métrique	Seuil minimum attendu	Commentaire
Couverture de code	65 % à 70 %	Priorité aux repositories MongoDB et à la logique métier
Passage des tests unitaires	100 %	Aucun merge autorisé si un test échoue

Passage des tests d'intégration	100 %	Obligatoire pour les fonctionnalités critiques
Validation API	Automatique	Validation des schémas via Ogen
Revue de code	1 reviewer minimum	Revue obligatoire avant fusion

Processus d'Exécution (Workflow)

1. Le développeur crée une branche depuis develop.
2. Il développe la fonctionnalité ou le correctif.
3. Il exécute les tests unitaires en local.
4. Il applique le formatage obligatoire (gofmt, Prettier, etc.).
5. Il ouvre une pull request.
6. GitHub Actions déclenche automatiquement les tests.
7. Un autre membre de l'équipe effectue une revue de code.
8. La pull request est fusionnée uniquement si la CI est valide et la revue approuvée.

DOCUMENTATION ONBOARDING

Readme

Le fichier **Readme** présente l'objectif de l'application, son positionnement ainsi que les fonctionnalités principales du projet Moozo. Il décrit ce que l'on construit et comment le produit répond aux besoins des organisateurs, des invités et des prestataires. On y retrouve aussi les informations de démonstration, les recrutements en cours ainsi que les auteurs du projet.

Contributing

Le fichier **Contributing** explique les règles pour participer au développement du projet. Il détaille le workflow Git à respecter, les standards de code, le format des commits, le processus de pull request ainsi que les bonnes pratiques de qualité logicielle afin de garantir un développement cohérent et maintenable.

CHOIX TECHNOLOGIQUES

Frontend – web

React TS

Le frontend web du projet Moozo est destiné aux invités afin de leur permettre d'accéder facilement aux informations de l'événement sans installation d'application mobile. Le choix de React avec TypeScript répond à cet objectif d'accessibilité, de performance et de maintenabilité.

React est une bibliothèque JavaScript open-source maintenue par Meta, utilisée pour construire des interfaces utilisateurs interactives basées sur des composants réutilisables. Selon l'enquête Stack Overflow Developer Survey 2024, React fait partie des technologies web les plus utilisées par les développeurs, ce qui garantit un écosystème mature et une large disponibilité de ressources techniques.

L'architecture component-based de React facilite le développement de ces fonctionnalités. Par exemple, le plan de table, la galerie de photos ou la liste des invités peuvent être développés sous forme de composants indépendants, réutilisables et facilement maintenables.

React utilise également un mécanisme appelé Virtual DOM, qui optimise la mise à jour de l'interface lorsque les données changent. Cela améliore la fluidité de l'expérience utilisateur dans des interfaces dynamiques où les contenus peuvent être mis à jour en temps réel (par exemple lors du partage de photos ou de l'ajout d'informations sur l'événement).

L'utilisation de TypeScript, développé par Microsoft, permet d'ajouter un typage statique au code JavaScript. Cela améliore la fiabilité de l'application et réduit les erreurs lors de la manipulation de données structurées comme les invités, les tables ou les informations d'événement. Le typage facilite également la maintenance et la collaboration sur un projet de taille importante.

Ainsi, React avec TypeScript constitue un choix adapté pour le frontend web de Moozo en offrant une interface performante, maintenable et accessible aux invités depuis n'importe quel navigateur sans installation préalable.

<https://survey.stackoverflow.co/2024/technology#2-web-frameworks-and-technologies>

Frontend - mobile

React Native

L'application mobile du projet Moozo est destinée principalement aux organisateurs et aux prestataires, qui doivent gérer l'événement directement depuis leur smartphone. Pour cela, le framework React Native avec TypeScript a été choisi.

React Native est un framework open-source développé par Meta permettant de créer des applications mobiles pour Android et iOS à partir de la même base de code JavaScript. Contrairement aux solutions hybrides basées sur du HTML, React Native utilise des composants natifs, ce qui permet d'obtenir des performances proches des applications mobiles natives.

React Native permet de développer ces fonctionnalités tout en réutilisant les concepts et une partie de la logique utilisés dans React (composants, gestion d'état, hooks). Cette cohérence technologique réduit la complexité du développement et facilite la maintenance du projet.

Comme pour le frontend web, l'utilisation de TypeScript améliore la qualité du code grâce au typage statique. Cela est particulièrement utile dans une application manipulant de nombreuses entités (événements, invités, tables, photos, prestataires), car les erreurs peuvent être détectées dès la phase de développement.

React Native dispose également d'un écosystème riche de bibliothèques permettant d'intégrer facilement des fonctionnalités nécessaires dans une application événementielle, comme l'accès à la caméra, les notifications ou la gestion des médias.

Ainsi, React Native avec TypeScript permet de développer une application mobile performante et multiplateforme, adaptée aux besoins des organisateurs et prestataires du projet Moozo.

Backend (API)

Nos choix se font dans l'objectif d'avoir une api rapide et claire, un code lisible et maintenable

Golang

Performance et efficacité

Go est un langage compilé qui produit un binaire natif, ce qui lui confère des performances proches du C sans la complexité de gestion mémoire manuelle. Pour une API REST, cela se

traduit par une faible latence et une consommation mémoire réduite, deux critères importants pour un service backend.

Lisibilité et maintenabilité

Go impose un style de code unique via gofmt et un ensemble de contraintes volontairement restrictives (pas de surcharge, pas d'héritage, gestion d'erreur explicite). Cela rend le code homogène et facilement lisible par tous les membres d'une équipe, ce qui est particulièrement pertinent dans un contexte de projet en groupe.

Simplicité du déploiement

Go compile en un binaire statique autonome, sans runtime externe à installer. Cela simplifie considérablement le déploiement et la conteneurisation.

Ogen

Approche OpenAPI First

Ogen adopte une approche *API First* : le fichier OpenAPI est la source de vérité unique à partir de laquelle le code serveur est entièrement généré (routing, validation, sérialisation). Cela garantit une cohérence permanente entre la documentation et l'implémentation.

Réduction du boilerplate

Le développeur n'implémente que la logique métier, sous forme d'une interface Go générée. Tout le code d'infrastructure HTTP est pris en charge par Ogen, ce qui réduit les erreurs et le temps de développement.

Sécurité par le compilateur

Si le spec OpenAPI évolue, le compilateur Go signale immédiatement les handlers manquants ou incompatibles. C'est une garantie structurelle de cohérence, plus fiable qu'une convention d'équipe.

Backend (IA)

Pourquoi l'intégrer dans un microservice séparé ?

- Le backend principal est en GO
- L'IA consomme beaucoup de CPU/RAM
- Les MàJ du service d'IA et de l'API ne sont pas synchrone

- Rollback/scalabilité indépendant

Python

- Ecosystème ML dominant
- Libs optimisées
- Enorme communauté

PyTorch

- Simple et intuitif
- Très utilisé en CV
- Debug facile
- Grosse communauté

Pourquoi pas TensorFlow ? Plus lourd et complexe, fait pour de gros projets avec beaucoup de stats

Base de données

MongoDB

Le projet Moozo manipule plusieurs types d'entités qui évoluent au cours du développement : utilisateurs, événements, invités, tables, photos, demandes de prestations, etc. Pour stocker ces données, nous avons retenu MongoDB.

MongoDB est une base de données orientée document : les données sont stockées sous forme de documents JSON (en BSON en interne). Ce modèle correspond bien aux objets manipulés côté API, ce qui simplifie le passage entre la base et le code applicatif sans passer par un mapping relationnel complexe.

Sur un projet en phase de conception, le modèle de données évolue régulièrement. Avec une base relationnelle, chaque modification de schéma implique une migration. MongoDB n'impose pas de schéma fixe au niveau de la base : ajouter un champ comme un statut RSVP, une préférence alimentaire ou un rôle prestataire ne demande pas de migration. La validation du schéma reste assurée côté API, ce qui garantit la cohérence des données sans bloquer les itérations.

Le modèle document est aussi adapté à des entités riches comme un événement, qui regroupe naturellement plusieurs informations liées (configuration, planning, liste d'invités, tables). Stocker ces informations dans un même document évite des jointures multiples et reste lisible à manipuler.

MongoDB dispose enfin d'un écosystème mature : drivers officiels pour Go et Python, outils d'indexation, sauvegardes et bonne intégration en environnement conteneurisé.

Pourquoi pas SQL (PostgreSQL / MySQL) ?

Une base SQL comme PostgreSQL impose un schéma strict et des migrations à chaque évolution du modèle. Cela apporte une garantie d'intégrité forte, mais ralentit les itérations sur un projet où la structure des entités n'est pas encore figée. Pour Moozo, la flexibilité de MongoDB et la rapidité de développement priment sur la rigidité d'un schéma relationnel.

Déploiement

Docker

Le déploiement de Moozo repose sur Docker afin d'assurer la cohérence entre les environnements de développement, de test et de production.

Docker permet d'embarquer une application avec l'ensemble de ses dépendances dans un conteneur isolé. Un service qui fonctionne sur la machine d'un développeur fonctionnera de la même manière sur le serveur, sans variation liée à la version du système, des bibliothèques ou de la configuration. Cela évite la classe d'erreurs typique du "ça marche chez moi".

L'architecture de Moozo se compose de plusieurs services : l'API en Go, le service d'IA en Python et la base MongoDB. Chacun est conteneurisé séparément, ce qui permet de les démarrer, mettre à jour ou redéployer indépendamment. Cette séparation est cohérente avec le choix d'isoler le service d'IA du backend principal évoqué plus haut, notamment pour gérer ses besoins en CPU/RAM et ses cycles de mise à jour propres.

Docker s'intègre également bien avec une chaîne CI/CD : la construction d'une image, son test puis son déploiement peuvent être automatisés de manière reproductible. Pour un projet d'équipe, cela simplifie aussi le lancement local — un docker-compose suffit à démarrer toute la stack sans avoir à configurer manuellement chaque dépendance sur sa machine.

Pourquoi pas Kubernetes ?

Kubernetes est conçu pour orchestrer des architectures distribuées à grande échelle : auto-scaling, haute disponibilité, gestion multi-nœuds, déploiements progressifs sur de

nombreux services. Sur un projet de la taille de Moozo, cette complexité n'est pas justifiée. La mise en place d'un cluster, des manifests, de l'ingress et de la gestion des secrets représenterait un surcoût important pour un bénéfice limité. Docker couvre les besoins actuels, et Kubernetes pourra être envisagé plus tard si la charge ou les besoins de scalabilité l'imposent.